On The Nature Of The Phenotype In Tree Genetic Programming: a review of Banzhaf & Bakurov 2024 [2]

Dineth Hirun Wickramasekara

University of Adelaide

1 Introduction

Genetic Programming (GP) is a branch of evolutionary computation, characterized by its use of non-linear representations such as *Tree structures and Graphs*, as opposed to the real valued vectors in ES or binary strings in GA [9]. It is primarily used for evolving programs, with applications in Machine Learning tasks such as classification, regression and feature engineering [2]. The use of Tree structures here allows GP to represent programs in a hierarchical manner where *'nodes'* represent operators and *'leaves'* represent constants or variables. The process of evolving solutions is done by applying genetic operators such as crossover and/or mutation to subtrees within the genotype. Consequentially, genotype trees can grow excessively large without a corresponding improvement in performance; often referred to as the '**bloat**' phenomenon [2].

Banzhaf & Bakurov [2] observed that since the phenotype encapsulates the actual executable behaviour, the ability to *examine* the phenotype for a specific genotype would facilitate a better understanding of the genotype-phenotype relationship in TGP, as well as provide insights for future research on new bloat control methods. In their research, they developed a simplification algorithm that extracts phenotypes by removing "semantically ineffective" code from genotype trees, resulting in a **size reduction of up to 20 folds**. This review will first briefly summarize their concepts, linking it to course content. Then, it will review its significance, potential applications, the methodology and arguments used.

2 Summary and new concepts

2.1 Genetic Programming and Bloat

Since the area of research is *Genetic Programming*, this paper primarily relates to the content in Week 4 of the course, and Chapter 6 of the textbook. As mentioned in the lecture[9], Genetic Programming utilizes Tree-based representation, along with the genetic operators of *recombination* by exchange of subtrees, and *mutation* by random changes in subtrees. As a result, the size of offspring can exceed that of the parents, and tree sizes in the population continue to grow throughout generations. This is referred to as *'Bloat'*.

Considering the common use of Fitness Based Selection in GP, the authors expand on the causes of bloat by quoting previous research by Tackett [6]: "the average growth in size is proportional to the underlying selection pressure, and bloat does not occur in cases where fitness is ignored" . Furthermore, they also highlight other research demonstrating that bloat is an *"evolutionary response"* aimed at safeguarding solutions from the destructive effects of genetic operators (mainly crossover), connecting to findings where these operators were more likely to deteriorate offspring fitness instead of increasing it [5].

They use a biological analogy, comparing bloat to "**non-coding DNA**" which refers to DNA *not involved* in providing instructions for creating proteins. It encapsulates 97-99% of total DNA, and was initially assumed as 'junk' providing no functional benefit [10]. However, it was later found that non-coding DNA helps with genomic rearrangements and acts as regulatory elements that control where and when genes are activated. Since evolutionary computation is primarily influenced by evolutionary biology, they use this analogy to suggest that similar to non-coding DNA, *bloat* or *non-coding regions of genotypes* may also provide to be useful in artificial evolution.

Considering this, it is also important to weigh the potential consequences of bloat in Genetic Programming. One such consequence mentioned is that it *increases* the amount of computer memory required to store a population, and the resources required to evaluate trees [2]. Since GP requires extremely large population sizes [9], this potentially makes the system unappealing for real-world applications. Moreover, excessively large genotype trees make it challenging, if not impossible, to visually inspect and interpret evolving solutions [2].

2.2 Genotype-phenotype mapping

As explained in the first lecture of the course [7], the phenotypic traits are behavior or physical differences that directly affect the response to the environment. The authors use the same basis of defining the phenotype as the structure that determines behavior. Genotype is the encoded representation of the solution [7], which can then be manipulated by genetic operators such mutation and crossover.

Banzhaf and Bakurov's [2] research pertains to extracting the phenotype from a genotype in order to interpret the solution. This is essentially a decoding process [7], different from the encoding processes we studied and considered in the course, particularly in Genetic Algorithms [8] and in assignment 1 where we encoded tours with permutation representations for the Travelling Salesman Problem. Here, the authors developed a simplification algorithm that derives the phenotype from each genotype under evolution, without significantly changing the behavior. They apply two types of simplification; one which preserves the behavior and therefore reveals the "**exact phenotype**", and one which slightly changes the behavior, leading to an "**approximate phenotype**" (see Fig. 1). They test the designed algorithm on 5 real-world regression problems, and introduce metrics and concepts beyond our course, such as "*Semantics Mean Absolute Deviation*" which quantifies the deviation between genotype and phenotype

semantics. A detailed review of their methodology will be done in the "Methodology" section (4) of this paper.

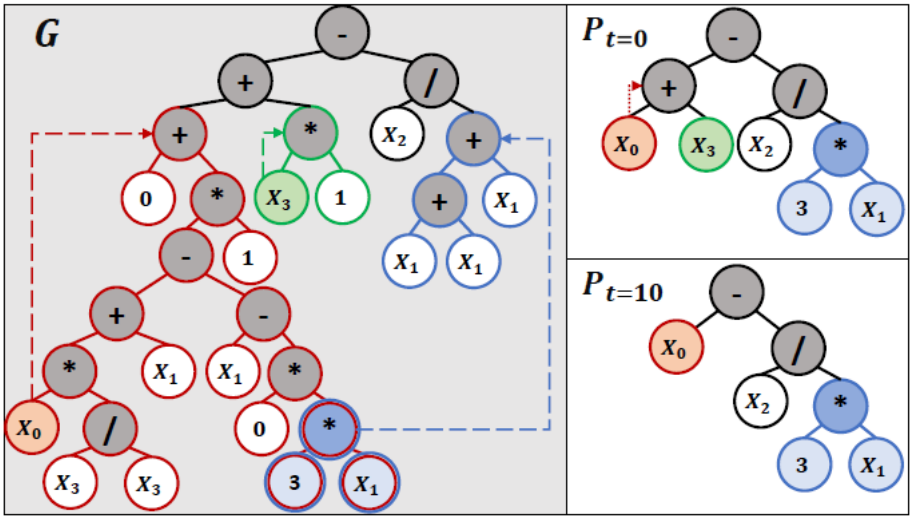

Fig. 1. A genotype (left) and its corresponding exact phenotype (top-right) and the approximate phenotype (bottom-right). Once you extract the semantically ineffective code from the genotype, its phenotype is revealed, which is comparatively smaller in size. Figure 2 of Bhanzaf & Bakurov [2]

3 Significance and Novelty

The core contribution of this paper is the novel simplification method that maps a genotype to its corresponding phenotype by extracting semantically ineffective code. As stated by the authors, it is *not meant to directly control bloat*, but rather serve as a tool to "look at" and understand evolving solutions without modifying behavior. Therefore, you cannot categorize it as, or directly compare it to bloat control methods such as imposing tree-depth limits or applying parsimony pressure [9]. However, it can be considered as a contribution that supports future research on the development of bloat control methods. Another potential real-world application of this would be in the realm of Explainable Artificial Intelligence [1]: in sensitive fields such as healthcare, security or legal sectors, being able to understand how solutions evolve may contribute towards building trust and confidence in using AI models.

Considering the realm of Tree Genetic Programming, their approach appears to be significantly unique. In the realm of Evolutionary Biology, methods and research into genotype-phenotype mapping already exists; for example the

"genome-wide CRISPR system" devised by Lian et al [4] aimed at investigating and engineering phenotypes for biotechnological applications. A "Boolean linear genetic programming algorithm" was also previously developed for genotype-phenotype mapping by Banzhaf et al [3], one of the same authors. However, it uses sequential representation and belongs to LGP (Linear Genetic Programming), which differs from Tree Genetic Programming and the tree structure representation used in this approach.

As explained by the "non-coding DNA" analogy, the paper also raises the topic of the *potential uses or advantages* of bloat, as opposed to only viewing it as an obstruction to performance. This can also be considered a significant contribution, as it prompts the need for research on the role of "redundant genetic material" in evolutionary algorithms. For the same reason, it offers a valuable perspective that should be considered when developing new-bloat control methods in the future.

4 Methodology

The simplification algorithm designed by the authors utilizes a **problem independent parameter 't'** to control the degree of simplification, enabling both exact and approximate phenotype extraction. This design choice introduces a valuable level of flexibility, allowing the algorithm to precisely eliminate subtrees that do not influence the phenotype when $\mathbf{t{=}0}$, or to remove subtrees that have minimal contributions to overall behavior for $\mathbf{t{>}0}$ (see Fig. 1). This feature gives their algorithm the ability to adapt to specific problem domains where different levels of simplification might be more effective. Overall, the authors have effectively explained the effect of this parameter on the final outputs, as well as the specific mechanisms by which it interacts with the simplification process and controls the approximation accuracy.

The experiment was conducted across 5 standard regression datasets, though the specific reason behind selecting these particular datasets is not mentioned. However, the datasets seem diverse considering the number of features, instances and target ranges, suggesting that the authors tested across varying complexities. Furthermore, the testing on regression problems raises the question of how the simplification technique would perform in classification and other GP applications. In terms of the hyper-parameters applied, the authors employed **two distinct settings**: one with a low mutation rate and high crossover rate (20%, 80%) for a *more exploitative search*, and another with a high mutation rate and low crossover rate (80%, 20%) for a more *explorative search*. Tournament selection with 4% pressure was used for parent selection, however the authors have not included the justification behind using this specific pressure value. This omission is significant, as they previously referenced findings that prove average growth in size in proportional to the selection pressure, therefore directly influencing bloat.

Another important decision made is that they applied **no limit to tree depth** in order to allow bloat to reveal itself without interference. While this

decision aligns with the purpose of the experiment to examine growth in genotype trees and its impact on corresponding phenotypes, it inevitably results in high computational demands that potentially limit the application of this algorithm in more complex-or large scale problems.

5 Argument and use of evidence

The logical flow from methodology to results in the paper is coherent, with the authors providing sub-sections and figures for size, semantic difference, diversity, fitness and structure. Since the experiment was done under two different settings (high crossover/low mutation and low crossover/high mutation) they provide a greatly detailed explanation of how each setting influenced the output of the algorithm. As observable from Fig. 2, the authors' initial claim of the algorithm being able to achieve a size reduction of up to 20 folds, from the genotype to the approximate phenotype, is proven.

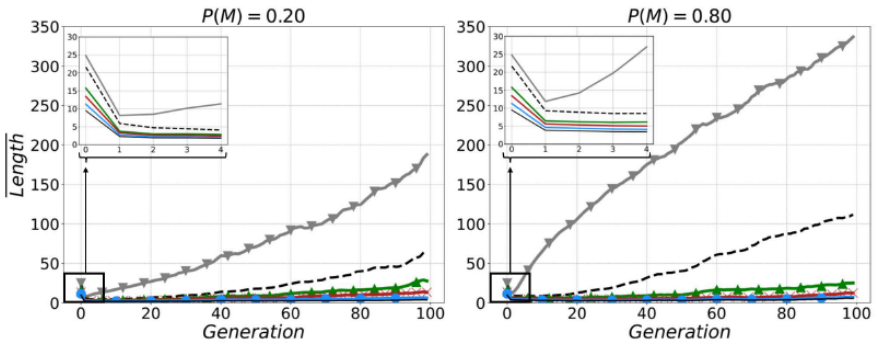

Fig. 2. The average population length over 100 generations, with a low mutation/high crossover setting (Left) and high mutation/low crossover setting (Right). The *genotype* is represented by the green line with downward-pointing triangles, the *exact phenotype* in black dashed lines, and the *approximate phenotype* by the red line with upward-pointing triangles. Fig. 3 of Banzhaf & Bakurov[2]

The use of a higher mutation rate, as observed in the right plot of Fig. 2, resulted in a larger growth in tree size. This finding contrasts with the previously referenced studies [5] that suggest crossover is more destructive in TGP. However, it supports the authors' suggestions on how bloat may be fostered as a *protective measure* against disruptive genetic operators, as increasing the mutation rate correlated to an increase in bloat[2]. The explanation given for the sudden drop in length during the first generation is only briefly addressed, linking it to the selection method used for the algorithm. This aspect could have been improved if the authors provided the reasoning behind the specific selection pressure used and how it affects simplification.

Considering the semantic mean absolute deviation (SMAD) between the genotype and the phenotypes (Fig. 2 of Banzhaf & Bakurov [2]), the problem independent parameter 't's effect on removing semantically ineffective code from the genotype, and therefore the algorithm's effectiveness is again demonstrated. An exponential decay in SMAD can be observed, with higher values of 't' (approximation) resulting in a stronger simplification and therefore higher SMAD.

While the effectiveness of the designed simplification algorithm in extracting the phenotype is *strongly proven* here, the high computational demand of the process (resulting from no-limitations on tree depth) still raises the question on its application in large-scale or real-time applications. The authors briefly conclude that it constitutes insights for future research on bloat control, and that they are focusing on improving the efficiency of the algorithm, with no further detail on its potential applications or how the reduced phenotypes translate to more interpretable models in practical scenarios.

Alternative interpretations, such as the possibility that the simplification *inadvertently biases* the evolutionary search towards certain solution structures, are not explored by the authors, representing another weakness of the paper. This potential bias could arise because the simplification algorithm might *favor* the retention of specific patterns or operators within the genotype trees, which as a result limits the diversity of solution structures the evolutionary process can explore. This could steer the search towards particular types of phenotypes, overlooking innovative or more complex structures that might offer better performance. Addressing these aspects would have strengthened the evaluation of their designed algorithm.

6 Conclusion

In summary, Bhanzaf and Bakurov present a unique simplification algorithm that effectively extracts phenotypes from genotypic trees by removing semantically ineffective code. They achieved a substantial size reduction of up to 20-folds in tree length, with a problem independent parameter that controls the degree of simplification. The paper presents strong evidence of this claim, and its effectiveness by an experiment that utilizes 5 regression datasets. The weaknesses of the paper are mainly regarding explanations behind certain decisions made in testing the algorithm; such as why they selected the specific regression datasets, or why they used a specific values for hyper-parameters (selection pressure etc.). It raises questions such as the effectiveness in the algorithm for other Machine Learning applications of GP (classification etc.), or on the potential bias that could make the algorithm favor the retention of certain operators, and therefore steer the evolutionary search towards specific solution structures. Overall, the paper makes a significant contribution to the area of Tree Genetic Programming, with valuable insights into the bloat phenomenon and genotype-phenotype relationships, and could be improved through more discussion into the rationale behind certain design decisions and practical applications.

References

1. Angelov, P.P., Soares, E.A., Jiang, R., Arnold, N.I., Atkinson, P.M.: Explainable artificial intelligence: an analytical review. WIREs Data Mining and Knowledge Discovery **11**(5), e1424 (2021). https://doi.org/https://doi.org/10.1002/widm.1424, https://wires.onlinelibrary.wiley.com/doi/abs/10.1002/widm.1424
2. Banzhaf, W., Bakurov, I.: On the nature of the phenotype in tree genetic programming (2024), https://arxiv.org/abs/2402.08011
3. Hu, T., Tomassini, M., Banzhaf, W.: A network perspective on genotype–phenotype mapping in genetic programming. Genetic Programming and Evolvable Machines **21**(3), 375–397 (Sep 2020). https://doi.org/10.1007/s10710-020-09379-0, https://doi.org/10.1007/s10710-020-09379-0
4. Lian, J., Schultz, C., Hamedi Rad, S., Zhao, H.: Multi-functional genome-wide crispr system for high throughput genotype–phenotype mapping. Nature Communications **10**, 5794 (12 2019). https://doi.org/10.1038/s41467-019-13621-4
5. Nordin, P., Francone, F., Banzhaf, W.: Explicitly Defined Introns and Destructive Crossover in Genetic Programming. In: Advances in Genetic Programming, Volume 2. The MIT Press (10 1996). https://doi.org/10.7551/mitpress/1109.003.0010, https://doi.org/10.7551/mitpress/1109.003.0010
6. Tackett, W.A.: Recombination, selection, and the genetic construction of computer programs. Ph.D. thesis, USA (1994), not available from Univ. Microfilms Int.
7. The University of Adelaide: Week 1: Introduction to evolutionary computation (2024)
8. The University of Adelaide: Week 2: Genetic algorithms (2024)
9. The University of Adelaide: Week 4: Evolutionary programming and genetic programming (2024)
10. Wu, A.S., Lindsay, R.K.: A survey of intron research in genetics. In: Voigt, H.M., Ebeling, W., Rechenberg, I., Schwefel, H.P. (eds.) Parallel Problem Solving from Nature — PPSN IV. pp. 101–110. Springer Berlin Heidelberg, Berlin, Heidelberg (1996)